

Liste des questions

Ci-dessous figure une série de questions types portant sur l'implémentation de votre scénario. Cette liste, fournie à titre indicatif, vous permettra de vous préparer à l'examen oral.

1. Expliquez une instruction précise dans l'un de vos Dockerfiles, du `gitlab-ci.yml` ou de votre script Terraform.
2. Commentez ou supprimez l'instruction `EXPOSE` de vos Dockerfile. Reconstituez l'image et redémarrez les conteneurs. Votre application fonctionne-t-elle toujours comme prévu ?
3. Modifiez le port utilisé dans l'application Django (par exemple, passez de `8000` à `8050`). Quels ajustements sont nécessaires pour que l'ensemble de votre développement fonctionne ?
4. Modifiez votre fichier `docker-compose.yml` en ajoutant ou supprimant l'option `depends_on` pour l'un de vos services. Après avoir apporté ce changement, lancez vos conteneurs et observez le comportement de l'ordre de démarrage des services. Expliquez l'impact de l'ajout ou de la suppression de `depends_on` sur le démarrage des services. Pourquoi et quand est-il important d'utiliser cette option dans un fichier `docker-compose.yml` ?
5. Ajoutez l'analyse SonarQube à votre pipeline GitLab CI pour analyser la qualité du code de votre projet. Modifiez le fichier `.gitlab-ci.yml` pour intégrer le job d'analyse SonarQube. Une fois l'analyse exécutée, expliquez les différentes étapes de cette intégration et montrez les résultats obtenus dans SonarQube. Discutez de l'impact de ces résultats sur l'amélioration de la qualité du code de votre projet.
6. Ajoutez une variable personnalisée à votre pipeline GitLab CI pour spécifier l'environnement de déploiement (par exemple, `DEPLOY_ENV`). Utilisez cette variable dans une condition pour contrôler l'exécution d'un job. Par exemple, créez un job qui ne se déclenche que si la variable `DEPLOY_ENV` est égale à `production`.
7. Modifiez votre fichier `.gitlab-ci.yml` pour que le pipeline ne se déclenche uniquement lorsqu'un tag spécifique, par exemple `présentation-2026`, est poussé sur le dépôt. Montrez les modifications apportées au fichier du pipeline, poussez le tag concerné, et vérifiez que le pipeline se déclenche bien uniquement dans ce cas. Expliquez comment fonctionne le déclenchement conditionnel dans GitLab CI.
8. Vous avez construit une image Docker dans un job de votre pipeline. Modifiez le pipeline pour permettre à un autre job de partager et d'utiliser cette image déjà buildée.
9. Sur un schéma représentant l'architecture de votre projet (avec les conteneurs et l'hôte), indiquez les ports utilisés par chaque service dans les conteneurs ainsi que sur votre machine hôte. Modifiez le mapping des ports : par exemple, changez le port interne d'un application pour `5050` et mappez-le sur le port `9090` sur votre machine hôte. Mettez à jour le schéma en conséquence.
10. Imaginons que votre enseignant souhaite utiliser votre script Terraform pour déployer l'infrastructure sur son propre compte Azure. Quelles modifications ou configurations devez-vous apporter au script et à son environnement pour qu'il puisse exécuter correctement le script sur son compte ? Montrez les étapes nécessaires et expliquez pourquoi chaque changement est nécessaire.
11. Exécutez votre script Terraform et identifiez les ressources Azure qui sont créées. Listez toutes les ressources déployées et montrez leur configuration dans Azure. Expliquez le rôle de chaque ressource dans l'infrastructure que vous avez mise en place et pourquoi elles sont nécessaires au bon fonctionnement de votre application.
12. Vous souhaitez changer le nom de toutes les images Docker générées en y ajoutant le suffixe `-ESI` (par exemple, `monimage-ESI`). Montrez et réalisez toutes les étapes nécessaires pour effectuer ce changement dans votre projet. Cela inclut la modification des Dockerfiles, la mise à jour du pipeline GitLab CI, et les ajustements dans Terraform si nécessaire. Après avoir appliqué ces changements, vérifiez que les nouvelles images sont correctement nommées et que le déploiement fonctionne toujours comme prévu.
13. Supprimez un de vos fichiers `.dockerignore`, reconstituez ensuite l'image Docker, lancez un conteneur à partir de cette image, puis explorez l'arborescence de fichiers à l'intérieur du conteneur. Montrez ce qui a été copié dans l'image et expliquez pourquoi la présence du fichier `.dockerignore` est importante.

Modifié le: vendredi 10 avril 2026, 07:52

